

Learn Low-Level Design in 25 Days: Day 3— Understanding Interfaces, Abstract Classes, and…

(Part of the series: Low-Level Design in 25 Days)

TechPreneur

Published October 16, 2025

Free: No

Read on Freedium · original

Contents

Why These Concepts Exist 0

The Core Concepts 0

Example 1: The Problem — Over-Inheritance 0

Example 2: The Solution — Interfaces and Abstract Classes 0

Step 1: Define a contract using an interface 0

Step 2: Provide shared behavior with an abstract class 0

Step 3: Implement concrete notifiers 0

Step 4: Use polymorphism 0

Visual Overview 0

Real-World Parallel: Slack's Notification System 0

Try It Yourself 0

Key Takeaways 0

Tomorrow's Lesson 0

The Day You Realize "Extends" Isn't Always the Answer

A few years back, I was mentoring a junior developer who proudly showed me his class hierarchy for a new payment module. There it was — a neat little tower of inheritance:

`PaymentProcessor` → `CardProcessor` → `CreditCardProcessor` → `VisaProcessor` → `OnlineVisaProcessor`.

I asked, "So, what happens when we add PayPal?" He paused. The tower shook.

That's the moment every engineer faces — when inheritance starts breaking rather than helping.

Welcome to **Day 3 of Learn Low-Level Design in 25 Days**. Today, we're tackling one of the most misunderstood yet foundational ideas in OOP: **Interfaces, Abstract Classes, and Inheritance**.



By the end, you'll know **when to extend**, **when to implement**, and **when to stop and rethink your design** — skills that separate clean architecture from messy dependency hell.

Why These Concepts Exist

Before we dive into code, let's clear the mental model.

In the real world, we organize things based on shared behavior and specialization.

- A Bird can fly.
- A Penguin is a bird but doesn't fly.
- A Bat flies but isn't a bird.

Try representing that only with inheritance, and you'll quickly realize — it's not enough. That's why **interfaces** and **abstract classes** exist: to express behavioral contracts and partial implementations — not just "is-a" relationships.

The Core Concepts

Let's break it down simply:

Concept	Purpose	Can Have Implementation?	Use When
Interface	Defines a contract or capability	❌ No	You want to define "what" an object can do
Abstract Class	Provides partial implementation	✅ Yes	You want to share base behavior but still keep flexibility
Inheritance (extends)	Reuse or specialize an existing class	✅ Yes	You have a clear "is-a" relationship

Example 1: The Problem — Over-Inheritance

Let's say we're building a notification system.

```
class Notification {
    void send() {
        System.out.println("Sending notification");
    }
}

class EmailNotification extends Notification {
    void sendEmail() {
        System.out.println("Sending Email");
    }
}

class SMSNotification extends Notification {
    void sendSMS() {
        System.out.println("Sending SMS");
    }
}
```

The problem?

- `Notification` isn't really a usable type — it's too generic.
- We're duplicating "send" logic everywhere.
- What if we later add Push or Slack notifications?

Example 2: The Solution — Interfaces and Abstract Classes

Let's fix this step-by-step.

Step 1: Define a contract using an interface

```
interface Notifier {
    void send(String message);
}
```

Step 2: Provide shared behavior with an abstract class

```
abstract class BaseNotifier implements Notifier {
    protected String sender;

    public BaseNotifier(String sender) {
        this.sender = sender;
    }

    public void log(String message) {
        System.out.println("[" + sender + "]: " + message);
    }
}
```

Step 3: Implement concrete notifiers

```
class EmailNotifier extends BaseNotifier {
    public EmailNotifier(String sender) { super(sender); }

    public void send(String message) {
        log("Email sent: " + message);
    }
}

class SMSNotifier extends BaseNotifier {
    public SMSNotifier(String sender) { super(sender); }

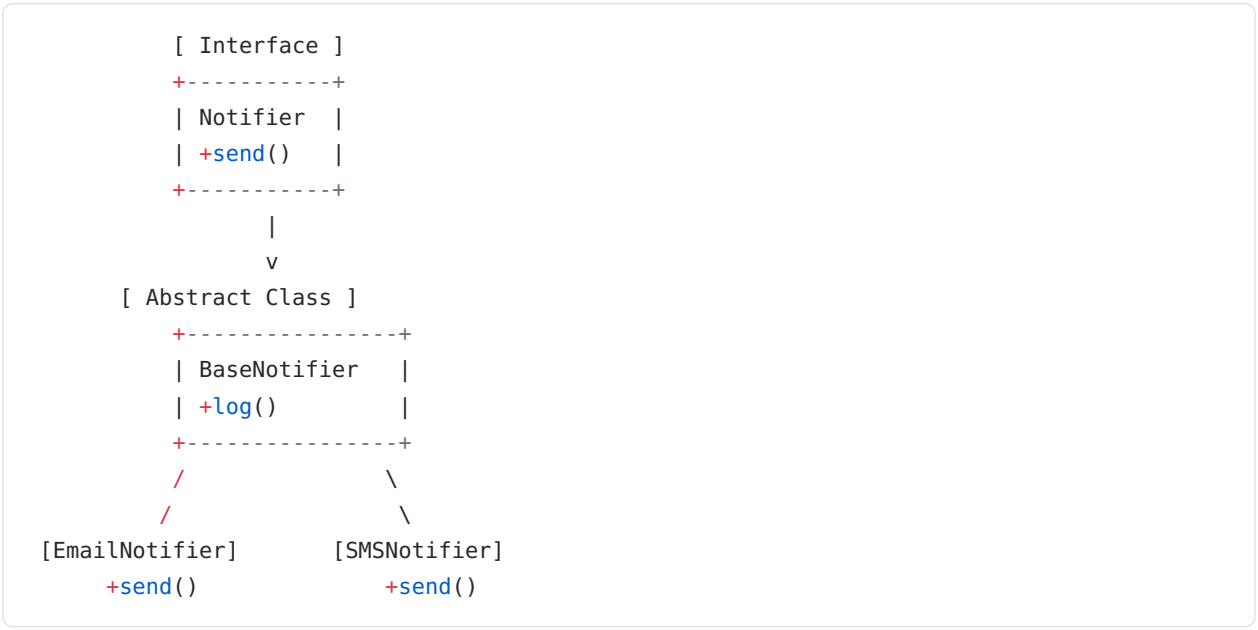
    public void send(String message) {
        log("SMS sent: " + message);
    }
}
```

Step 4: Use polymorphism

```
Notifier notifier = new SMSNotifier("System");
notifier.send("Server is down!");
```

Now, you can easily add a `SlackNotifier` or `PushNotifier` — without touching any existing code. That's **open for extension, closed for modification** (remember SOLID from Day 2?).

Visual Overview



Real-World Parallel: Slack's Notification System

Slack doesn't hardcode how to "send" a message. There's a common interface for "MessageSink" — something that can receive messages. The system doesn't care how messages are sent — via mobile, email, webhook, or desktop. Each medium implements the same contract differently.

That's how you scale a system from one notifier to ten without rewriting core logic.

Try It Yourself

Here are two small challenges to test your understanding:

1. **Design Challenge:** Imagine you're building a payment system with `CardPayment`, `UPIPayment`, and `CryptoPayment`.

- Which part should be an interface?
- Which can be an abstract class?

2. Code Challenge: Implement a `ReportGenerator` system where each generator (PDF, CSV, Excel) implements a `generate()` method, but they all share a common `validateData()` step.

Key Takeaways

- **Interfaces define "what"**, not "how."
- **Abstract classes share "how"**, but still leave room for customization.

- **Inheritance should model real "is-a" relationships**, not just for code reuse.
- Overusing inheritance leads to brittle, tightly coupled systems.
- Favor composition and interfaces for flexibility and testability.

Tomorrow's Lesson

Tomorrow, we'll explore **Composition vs Inheritance** — the design decision that can make or break a codebase. We'll see how modern systems like Uber and Spotify prefer "has-a" over "is-a" to stay flexible and scalable.

 Follow the full series here: Learn Low-Level Design in 25 Days — A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)